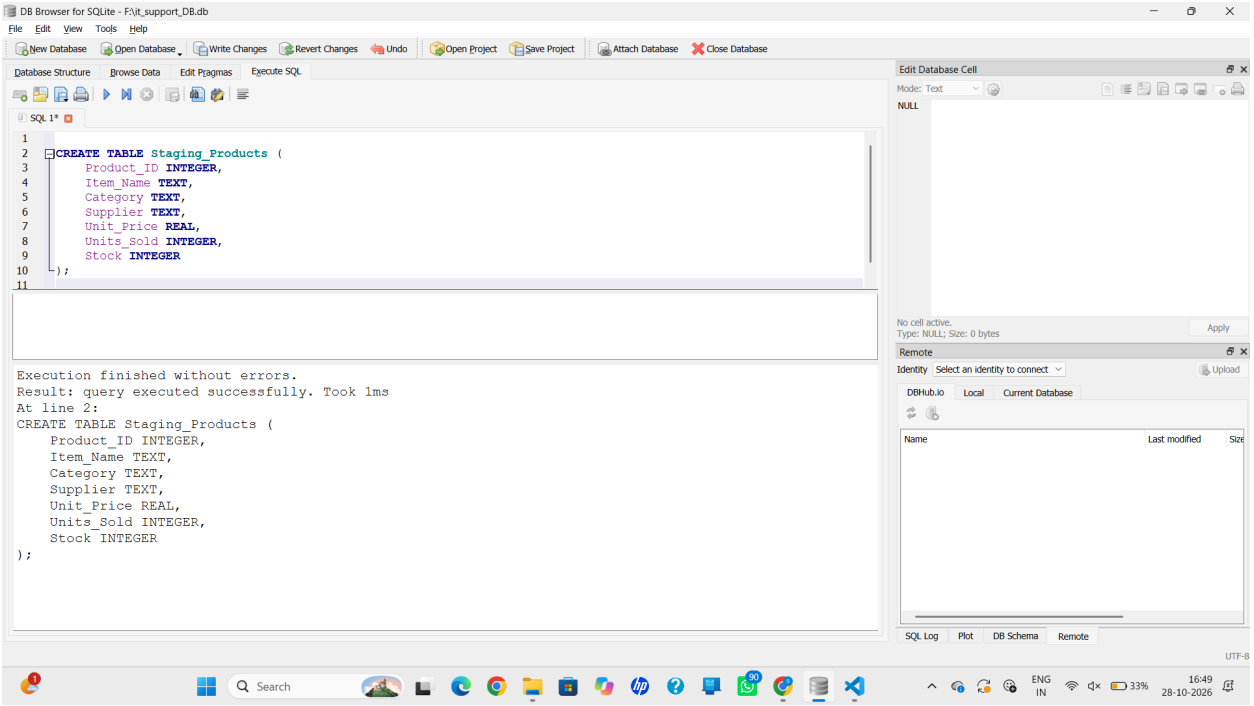
Dataset 2 Documentation

Create the staging table

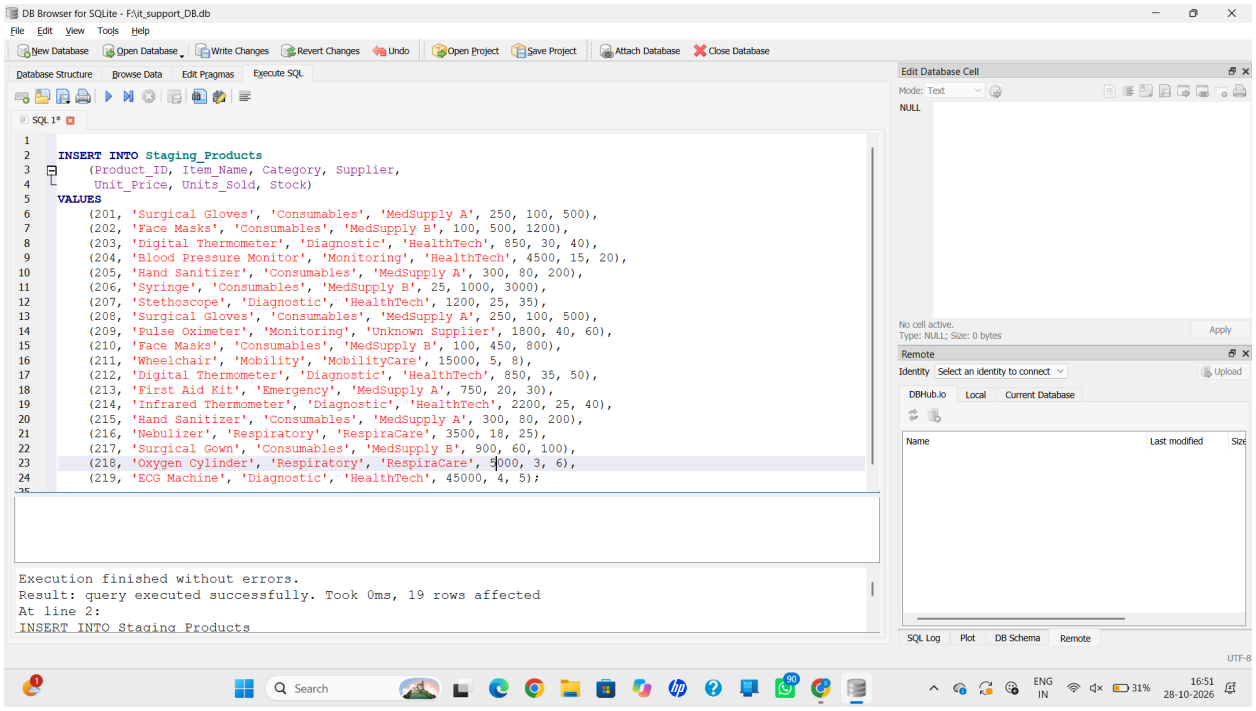
```
CREATE TABLE Staging_Products (
    Product_ID INTEGER,
    Item_Name TEXT,
    Category TEXT,
    Supplier TEXT,
    Unit_Price REAL,
    Units_Sold INTEGER,
    Stock INTEGER
);
```



Insert the dataset values

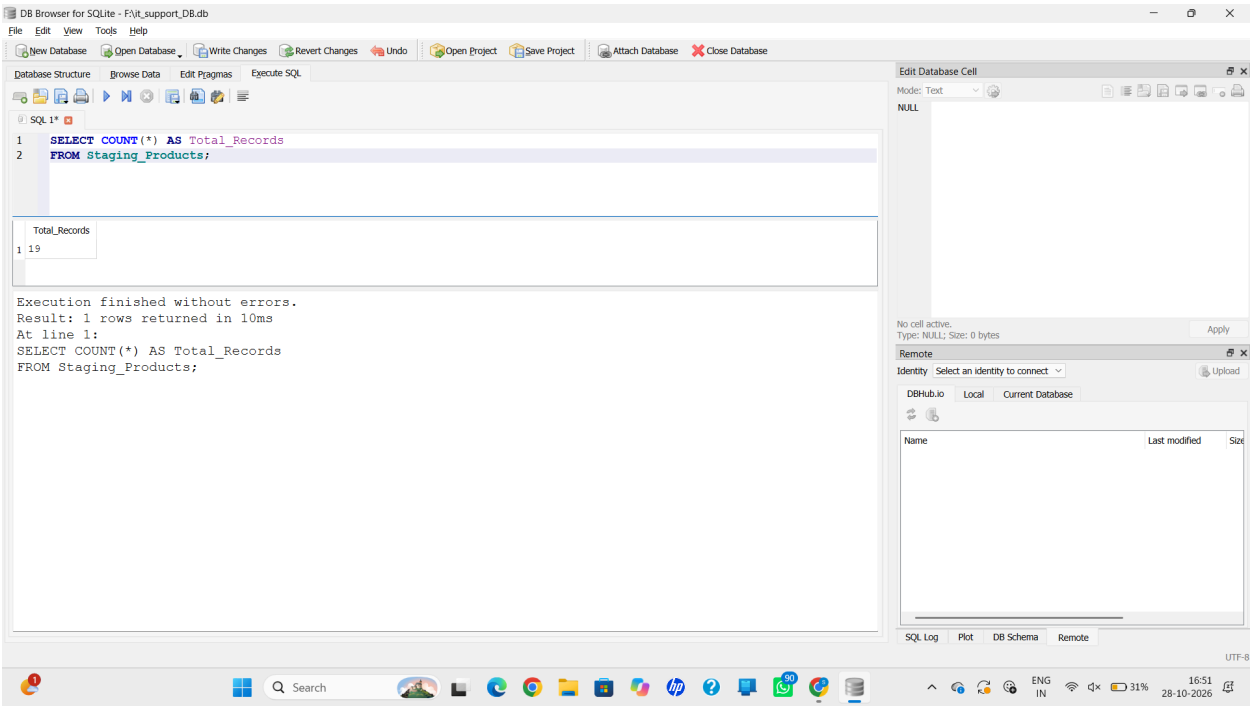
```
INSERT INTO Staging_Products
    (Product_ID, Item_Name, Category, Supplier,
     Unit_Price, Units_Sold, Stock)
VALUES
    (201, 'Surgical Gloves', 'Consumables', 'MedSupply A', 250, 100, 500),
    (202, 'Face Masks', 'Consumables', 'MedSupply B', 100, 500, 1200),
    (203, 'Digital Thermometer', 'Diagnostic', 'HealthTech', 850, 30, 40),
    (204, 'Blood Pressure Monitor', 'Monitoring', 'HealthTech', 4500, 15, 20),
    (205, 'Hand Sanitizer', 'Consumables', 'MedSupply A', 300, 80, 200),
    (206, 'Syringe', 'Consumables', 'MedSupply B', 25, 1000, 3000),
```

(207, 'Stethoscope', 'Diagnostic', 'HealthTech', 1200, 25, 35),
(208, 'Surgical Gloves', 'Consumables', 'MedSupply A', 250, 100, 500),
(209, 'Pulse Oximeter', 'Monitoring', 'Unknown Supplier', 1800, 40, 60),
(210, 'Face Masks', 'Consumables', 'MedSupply B', 100, 450, 800),
(211, 'Wheelchair', 'Mobility', 'MobilityCare', 15000, 5, 8),
(212, 'Digital Thermometer', 'Diagnostic', 'HealthTech', 850, 35, 50),
(213, 'First Aid Kit', 'Emergency', 'MedSupply A', 750, 20, 30),
(214, 'Infrared Thermometer', 'Diagnostic', 'HealthTech', 2200, 25, 40),
(215, 'Hand Sanitizer', 'Consumables', 'MedSupply A', 300, 80, 200),
(216, 'Nebulizer', 'Respiratory', 'RespiraCare', 3500, 18, 25),
(217, 'Surgical Gown', 'Consumables', 'MedSupply B', 900, 60, 100),
(218, 'Oxygen Cylinder', 'Respiratory', 'RespiraCare', NULL, 3, 6),
(219, 'ECG Machine', 'Diagnostic', 'HealthTech', 45000, 4, 5);



Count the records :

SELECT COUNT(*) AS Total_Records
FROM Staging_Products;



Create the three main tables

We'll use Categories, Suppliers, and Products. Because Product_ID 201 occurs twice, the Products table uses a separate primary key, product_row_id, so both original records can be retained.

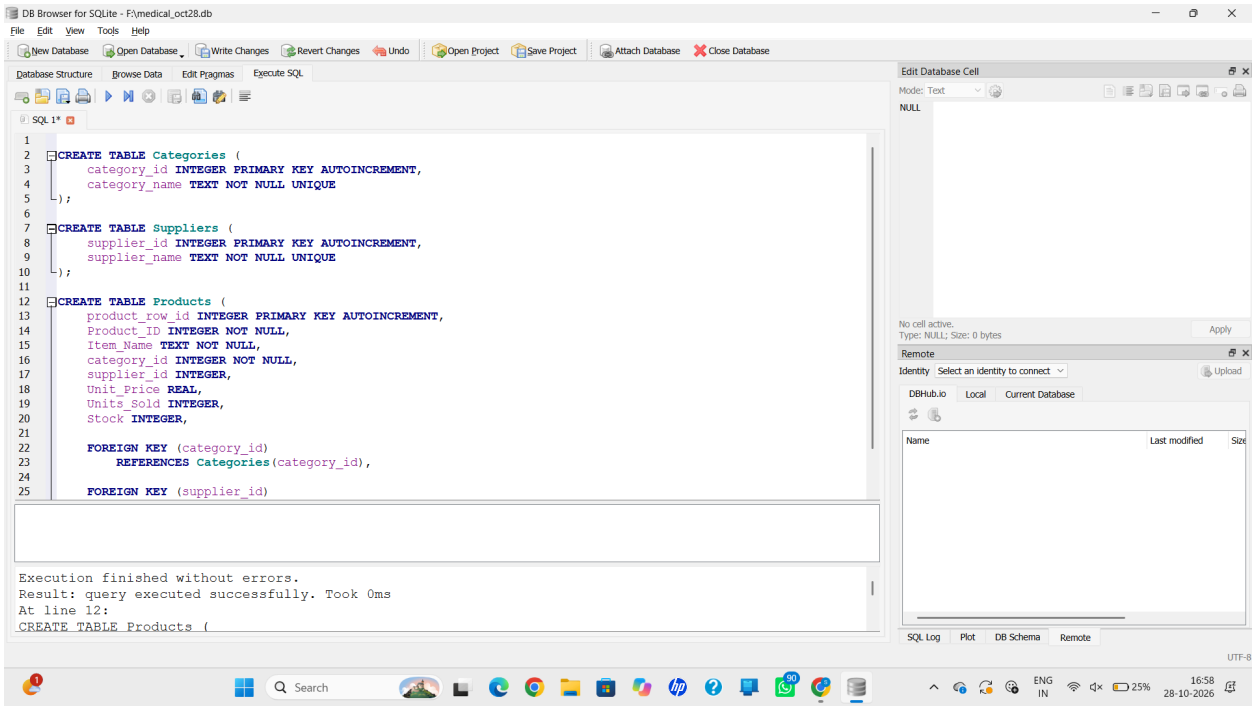
```
CREATE TABLE Categories (
    category_id INTEGER PRIMARY KEY AUTOINCREMENT,
    category_name TEXT NOT NULL UNIQUE
);

CREATE TABLE Suppliers (
    supplier_id INTEGER PRIMARY KEY AUTOINCREMENT,
    supplier_name TEXT NOT NULL UNIQUE
);
```

```
CREATE TABLE Products (
    product_row_id INTEGER PRIMARY KEY AUTOINCREMENT,
    Product_ID INTEGER NOT NULL,
    Item_Name TEXT NOT NULL,
    category_id INTEGER NOT NULL,
    supplier_id INTEGER,
    Unit_Price REAL,
    Units_Sold INTEGER,
    Stock INTEGER,

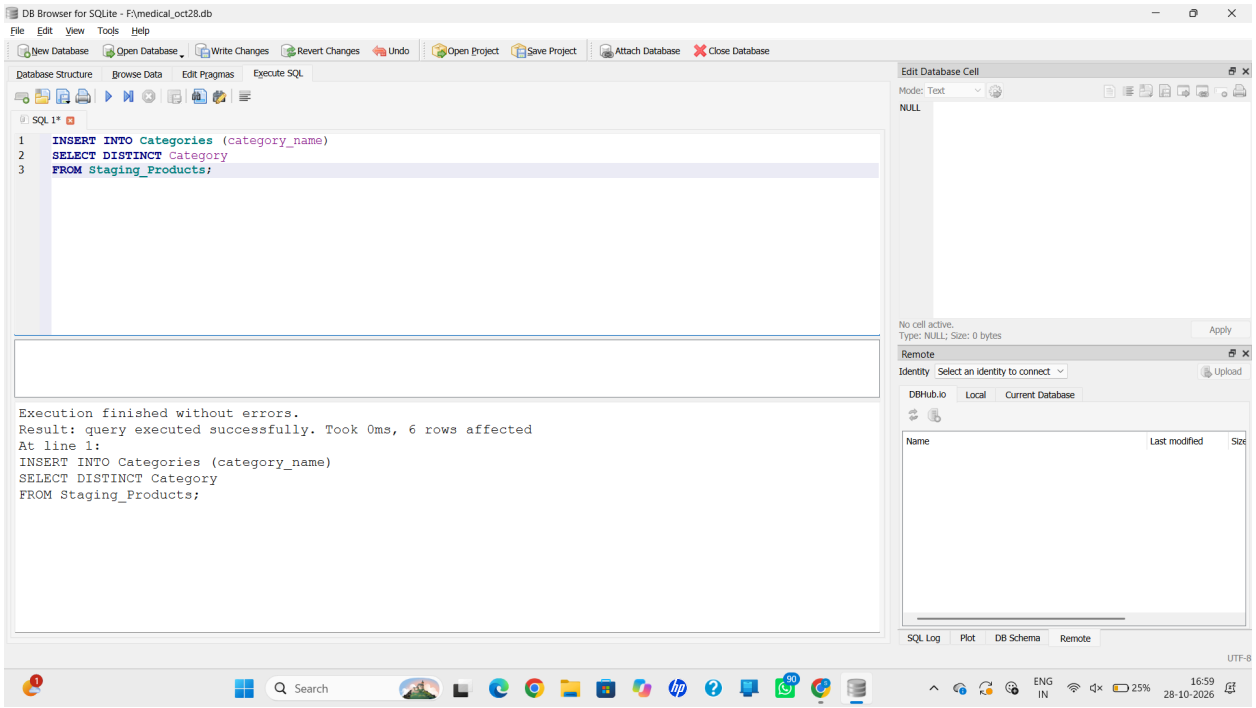
    FOREIGN KEY (category_id)
        REFERENCES Categories(category_id),

    FOREIGN KEY (supplier_id)
        REFERENCES Suppliers(supplier_id)
);
```



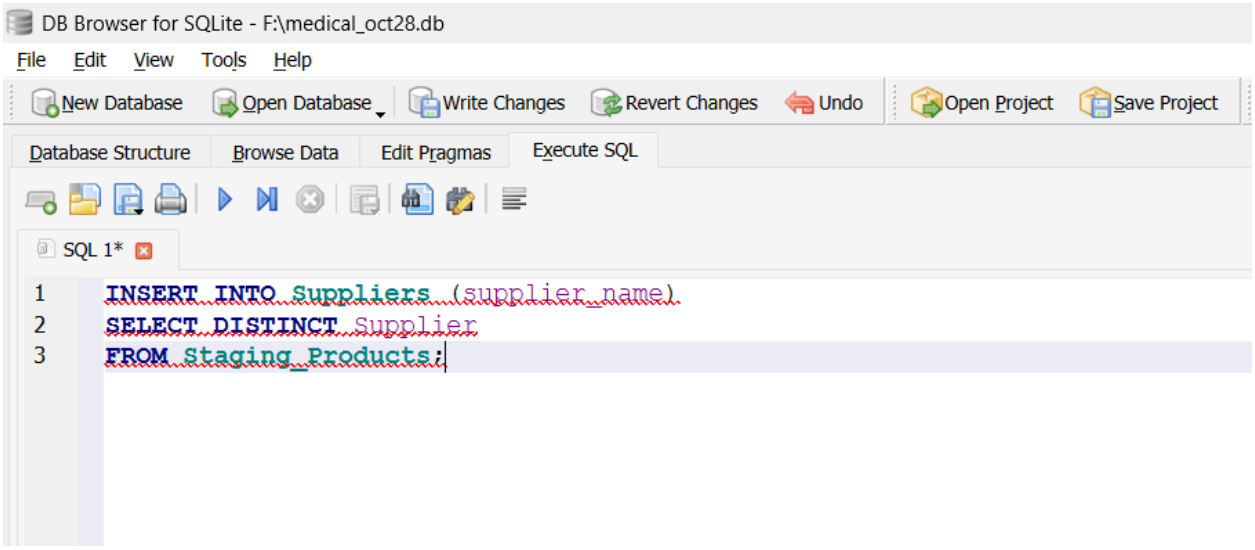
Populate Categories:

INSERT INTO Categories (category_name)
SELECT DISTINCT Category
FROM Staging_Products;



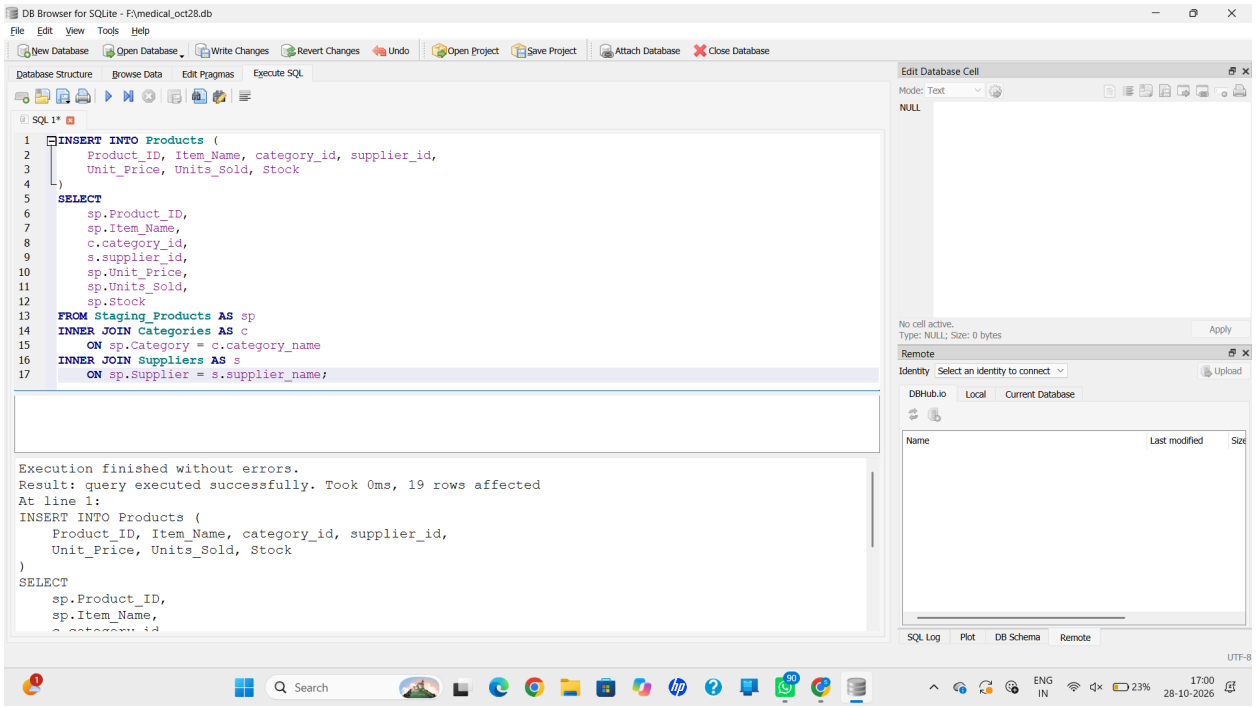
Populate Suppliers:

```
INSERT INTO Suppliers (supplier_name)
SELECT DISTINCT Supplier
FROM Staging_Products;
```

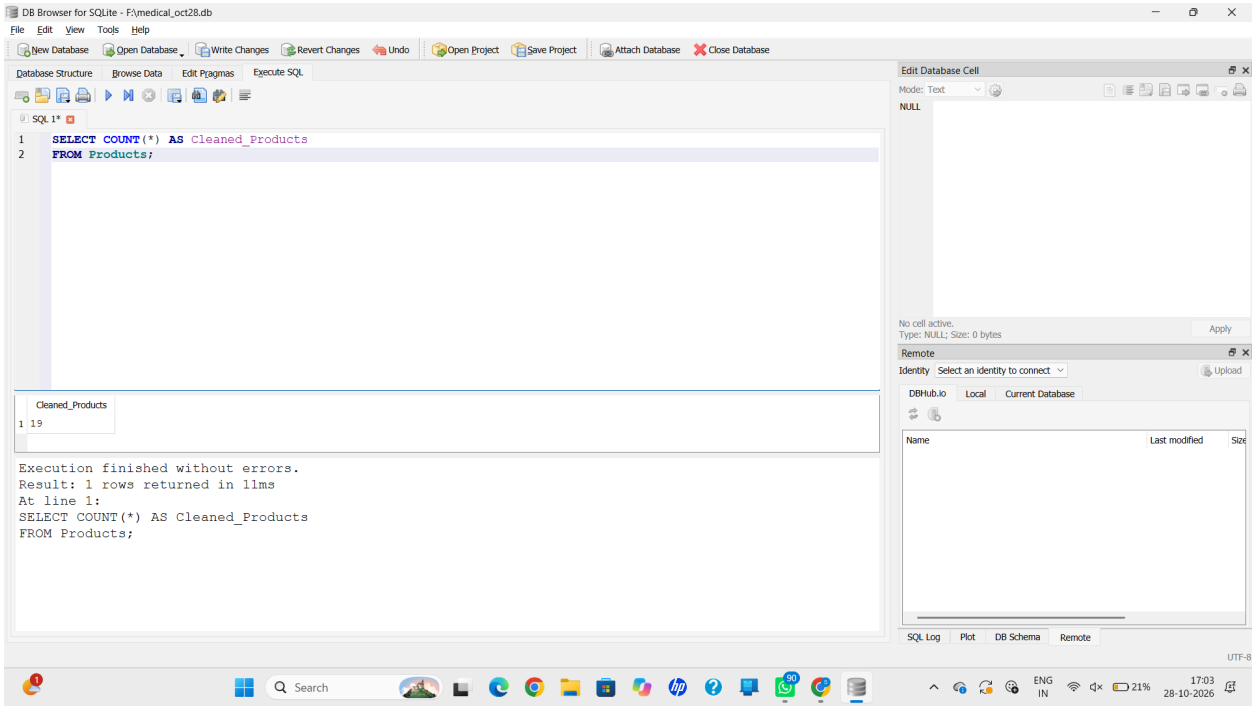


Populate Products:

```
INSERT INTO Products (
    Product_ID, Item_Name, category_id, supplier_id,
    Unit_Price, Units_Sold, Stock
)
SELECT
    sp.Product_ID,
    sp.Item_Name,
    c.category_id,
    s.supplier_id,
    sp.Unit_Price,
    sp.Units_Sold,
    sp.Stock
FROM Staging_Products AS sp
INNER JOIN Categories AS c
    ON sp.Category = c.category_name
INNER JOIN Suppliers AS s
    ON sp.Supplier = s.supplier_name;
```

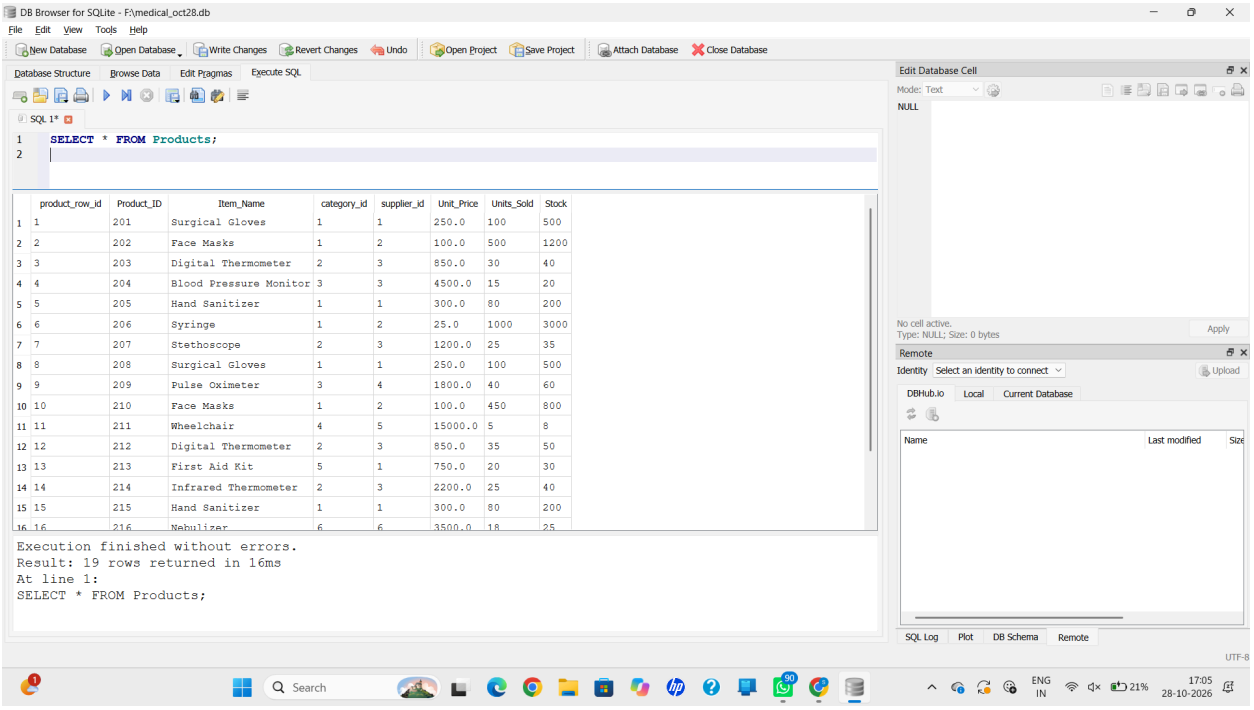


SELECT COUNT(*) AS Cleaned_Products
FROM Products;



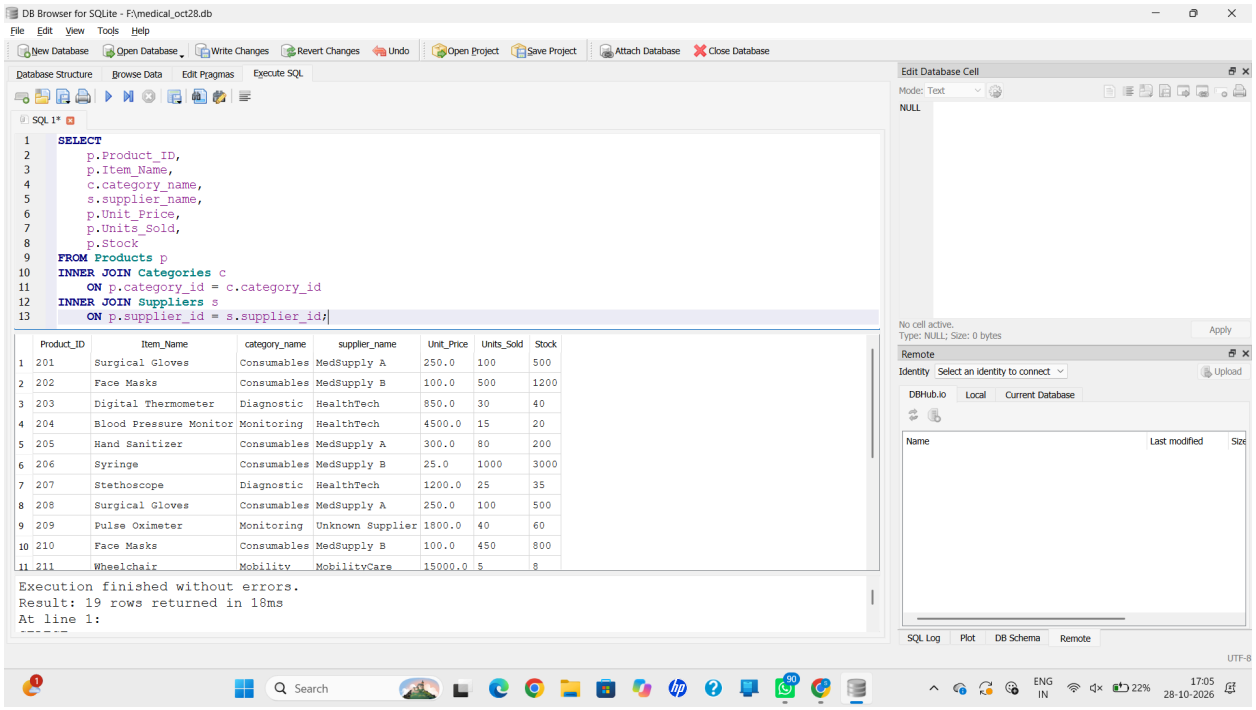
1: Display all products

SELECT * FROM Products;



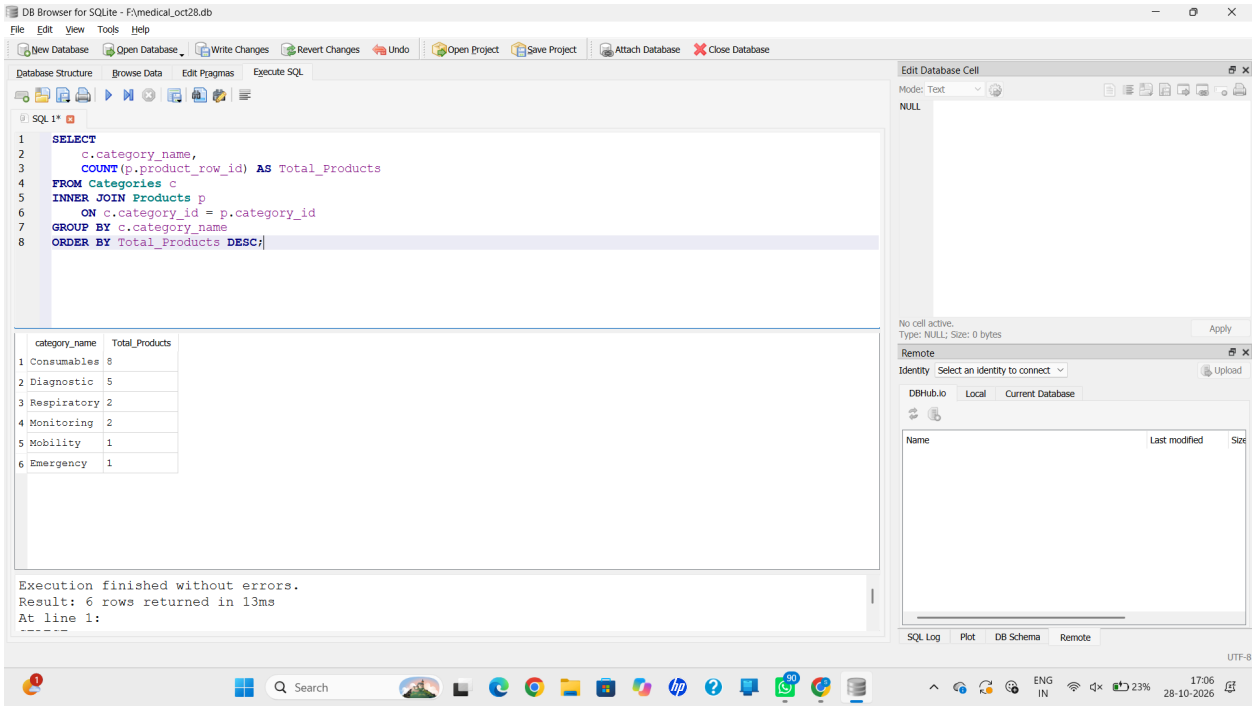
2: INNER JOIN — Products with categories and suppliers

```
SELECT
    p.Product_ID,
    p.Item_Name,
    c.category_name,
    s.supplier_name,
    p.Unit_Price,
    p.Units_Sold,
    p.Stock
FROM Products p
INNER JOIN Categories c
    ON p.category_id = c.category_id
INNER JOIN Suppliers s
    ON p.supplier_id = s.supplier_id;
```



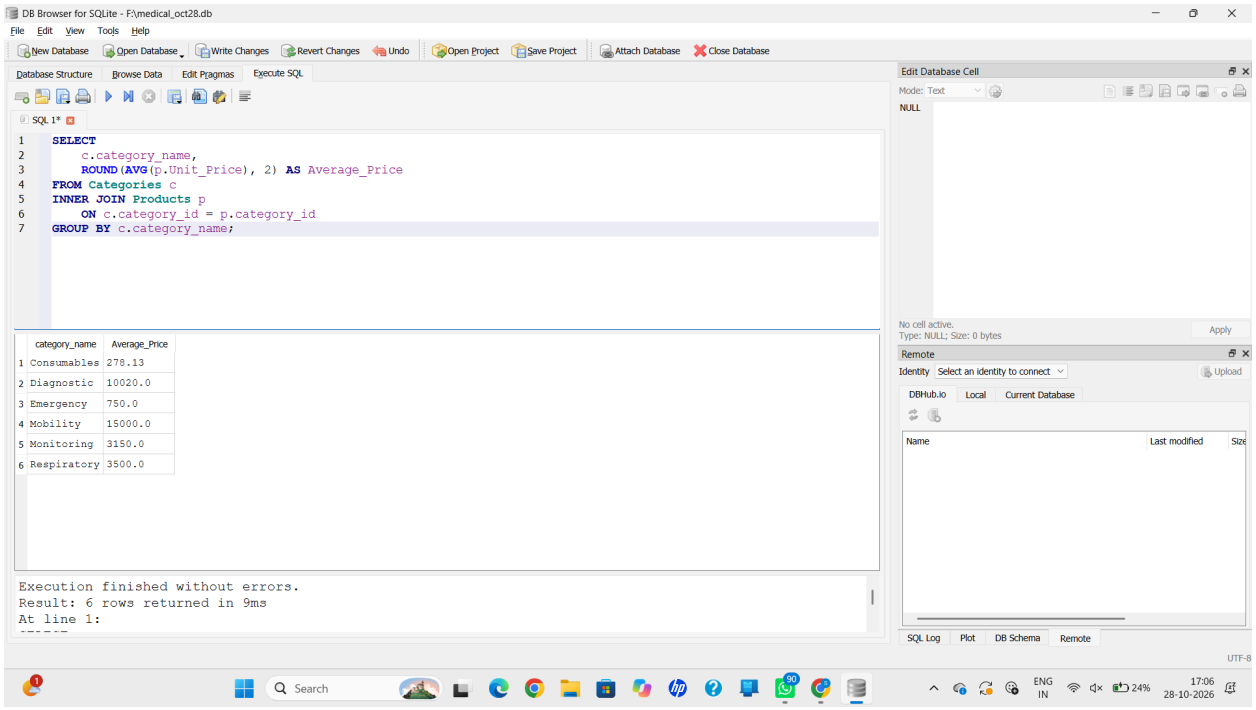
3: GROUP BY — Count products by category

```
SELECT
   c.category_name,
   COUNT(p.product_row_id) AS Total_Products
FROM Categories c
INNER JOIN Products p
   ON c.category_id = p.category_id
GROUP BY c.category_name
ORDER BY Total_Products DESC;
```


DB Browser for SQLite - F:\medical_oct28.db
File Edit View Tools Help
New Database Open Database Write Changes Revert Changes Undo Open Project Save Project Attach Database Close Database
Database Structure Browse Data Edit Pragmas Execute SQL
SQL 1*
1 SELECT
2 c.category_name,
3 COUNT(p.product_row_id) AS Total_Products
4 FROM Categories c
5 INNER JOIN Products p
6 ON c.category_id = p.category_id
7 GROUP BY c.category_name
8 ORDER BY Total_Products DESC;
category_name Total_Products
1 Consumables 8
2 Diagnostic 5
3 Respiratory 2
4 Monitoring 2
5 Mobility 1
6 Emergency 1
Execution finished without errors.
Result: 6 rows returned in 13ms
At line 1:
Edit Database Cell
Mode: Text
NULL
No cell active.
Type: NULL; Size: 0 bytes
Apply
Remote
Identity Select an identity to connect
Upload
DBHub.io Local Current Database
Name Last modified Size
SQL Log Plot DB Schema Remote
UTF-8
Search
ENG IN
23%
17:06
28-10-2026

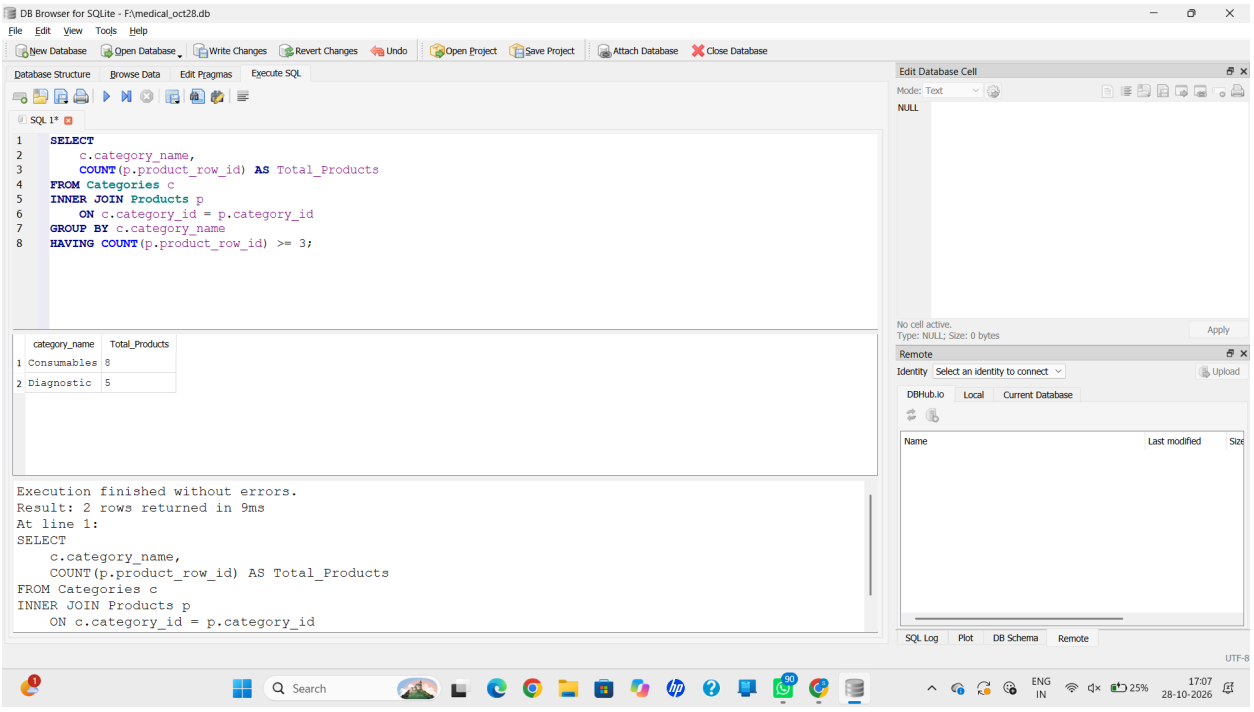
4: GROUP BY — Average price by category

```
SELECT
    c.category_name,
    ROUND(AVG(p.Unit_Price), 2) AS Average_Price
FROM Categories c
INNER JOIN Products p
    ON c.category_id = p.category_id
GROUP BY c.category_name;
```



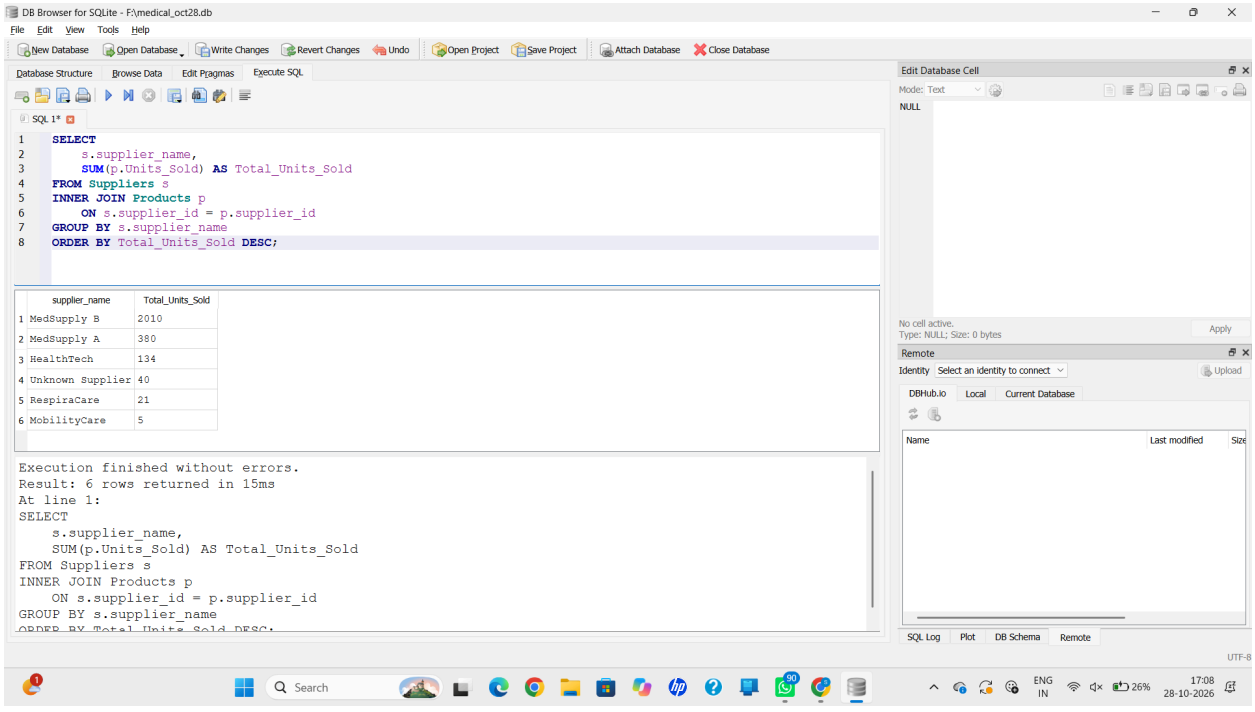
5: HAVING — Categories with at least 3 products

```
SELECT
    c.category_name,
    COUNT(p.product_row_id) AS Total_Products
FROM Categories c
INNER JOIN Products p
    ON c.category_id = p.category_id
GROUP BY c.category_name
HAVING COUNT(p.product_row_id) >= 3;
```



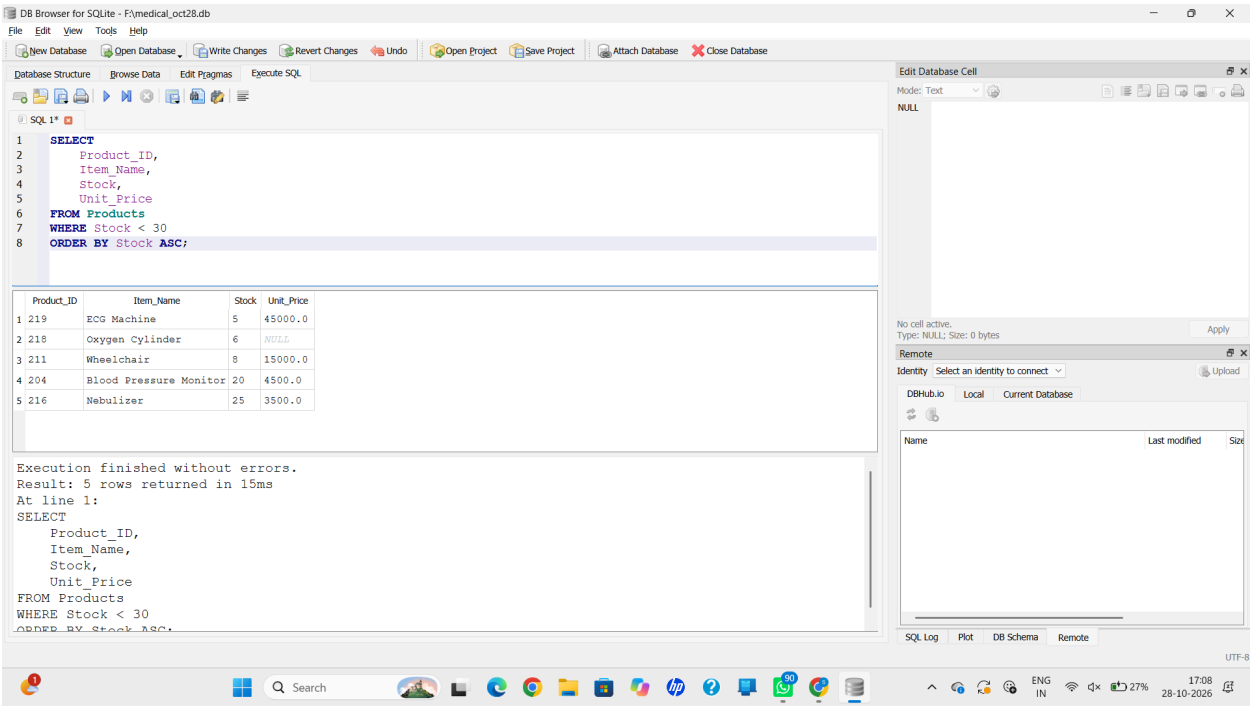
6: GROUP BY — Total units sold by supplier

```
SELECT
    s.supplier_name,
    SUM(p.Units_Sold) AS Total_Units_Sold
FROM Suppliers s
INNER JOIN Products p
    ON s.supplier_id = p.supplier_id
GROUP BY s.supplier_name
ORDER BY Total_Units_Sold DESC;
```



7: WHERE — Find products with low stock

```
SELECT
   Product_ID,
   Item_Name,
   Stock,
   Unit_Price
FROM Products
WHERE Stock < 30
ORDER BY Stock ASC;
```


DB Browser for SQLite - F:\medical_oct28.db
File Edit View Tools Help
New Database Open Database Write Changes Revert Changes Undo Open Project Save Project Attach Database Close Database
Database Structure Browse Data Edit Pragmas Execute SQL
SQL 1*
1 SELECT
2 Product_ID,
3 Item_Name,
4 Stock,
5 Unit_Price
6 FROM Products
7 WHERE Stock < 30
8 ORDER BY Stock ASC;
Product_ID Item_Name Stock Unit_Price
1 219 ECG Machine 5 45000.0
2 218 Oxygen Cylinder 6 NULL
3 211 Wheelchair 8 15000.0
4 204 Blood Pressure Monitor 20 4500.0
5 216 Nebulizer 25 3500.0
Execution finished without errors.
Result: 5 rows returned in 15ms
At line 1:
SELECT
Product_ID,
Item_Name,
Stock,
Unit_Price
FROM Products
WHERE Stock < 30
ORDER BY Stock ASC;
Edit Database Cell
Mode: Text
NULL
No cell active.
Type: NULL; Size: 0 bytes
Apply
Remote
Identity Select an identity to connect
Upload
DBHub.io Local Current Database
Name Last modified Size
SQL Log Plot DB Schema Remote
UTF-8
Search
ENG IN
27%
17:08
28-10-2026

8: Find the top 5 products by estimated sales revenue

```
SELECT
    Product_ID,
    Item_Name,
    Unit_Price,
    Units_Sold,
    (Unit_Price * Units_Sold) AS Total_Revenue
FROM Products
WHERE Unit_Price IS NOT NULL
ORDER BY Total_Revenue DESC
LIMIT 5;
```

